

Explique as principais estruturas de dados do Python e apresente exemplos de situações em que listas, tuplas e dicionários podem ser utilizadas.

Thoughts

Principais Estruturas de Dados do Python

No Python, as estruturas de dados nativas são utilizadas para agrupar e organizar coleções de informações na memória. As três principais estruturas de dados para análise e automação de dados são as Listas, as Tuplas e os Dicionários.

1. Listas (list)

O que é e características principais

Uma Lista é uma sequência ordenada e mutável de elementos.

Ordenada: Cada elemento possui um índice numérico fixo baseado na sua posição, iniciando em 0.

Mutável: É possível adicionar, remover ou modificar elementos sem alterar a identidade da lista na memória.

Sintaxe: Declarada utilizando colchetes [] com itens separados por vírgula.

Cenários de uso prático

Armazenar histórico de vendas diárias ou lançamentos financeiros de um período.

Acumular nomes de arquivos encontrados em um diretório para processamento.

Manter filas de tarefas a serem executadas em sequência.

Exemplo Prático

```
# Criando uma lista de vendas financeiras
vendas =

# 1. Adicionando uma nova venda no final da lista
vendas.append(1200)

# 2. Modificando a primeira venda (índice 0)
vendas[0] = 1100

# 3. Calculando a soma total das vendas
total_vendas = sum(vendas)

print("Vendas atualizadas:", vendas)
print("Total de vendas:", total_vendas)
```

Explicação passo a passo do código:

`vendas = [1000, 1500, ...]`: Inicializa a lista armazenando quatro números na memória.

`vendas.append(1200)`: Executa o método `.append()`, inserindo o valor 1200 ao final da lista existente.

`vendas[0] = 1100`: Acessa a primeira posição (índice 0) e substitui o valor original 1000 por 1100.

`sum(vendas)`: Utiliza a função embutida `sum()` para somar todos os elementos numéricos contidos na lista.

2. Tuplas (tuple)

O que é e características principais

Uma Tupla é uma sequência ordenada e imutável.

Imutável: Após a sua criação, seus valores não podem ser alterados, adicionados ou removidos.

Desempacotamento (Unpacking): Permite atribuir múltiplos valores retornados de uma só vez para variáveis individuais.

Sintaxe: Declarada utilizando parênteses () com itens separados por vírgula.

Cenários de uso prático

Definir configurações fixas do sistema que não devem ser alteradas acidentalmente (como dimensões de tela ou coordenadas).

Retornar múltiplos valores a partir de uma função (como taxas ou impostos calculados separadamente).

Exemplo Prático

```
# 1. Função que calcula e retorna múltiplos impostos
def calcular_impostos(valor_venda):
    ir = valor_venda * 0.15
    iss = valor_venda * 0.05
    return ir, iss # Retorna automaticamente uma tupla (ir, iss)

# 2. Chamada da função e desempacotamento dos resultados
imposto_ir, imposto_iss = calcular_impostos(2000)

print("Imposto de Renda (IR):", imposto_ir)
print("Imposto sobre Serviços (ISS):", imposto_iss)
```

Explicação passo a passo do código:

`return ir, iss`: O Python empacota automaticamente os dois valores devolvidos em uma tupla (`ir, iss`).

`imposto_ir, imposto_iss = calcular_impostos(...)`: Ocorre o desempacotamento (unpacking), onde o primeiro elemento da tupla é atribuído a `imposto_ir` e o segundo a `imposto_iss`.

3. Dicionários (dict)

O que é e características principais

Um Dicionário é uma coleção mapeada de pares chave-valor (key-value).

Acesso por Chave: Em vez de índices numéricos, os valores são acessados diretamente pelo nome da chave correspondente.

Chaves Únicas: Cada chave no dicionário é única; ao atribuir um novo valor a uma chave existente, o valor anterior é sobrescrito.

Sintaxe: Declarado utilizando chaves {} com pares no formato chave: valor.

Cenários de uso prático

Mapear produtos aos seus respectivos preços ou quantidade em estoque.

Representar estruturas complexas oriundas de APIs e arquivos JSON.

Consolidar métricas ou faturamentos agrupados por filial/vendedor.

Exemplo Prático

```
# 1. Criando um dicionário de preços de produtos
```

```
precos_produtos = {  
    "celular": 1500,  
    "camera": 1000,  
    "monitor": 2000  
}
```

```
# 2. Consultando o preço do "celular" diretamente pela chave
```

```
preco_celular = precos_produtos["celular"]
```

```
# 3. Adicionando um novo produto no dicionário
```

```
precos_produtos["televisao"] = 3000
```

```
print("Preço do celular:", preco_celular)
```

```
print("Catálogo atualizado:", precos_produtos)
```

Explicação passo a passo do código:

`precos_produtos = {"celular": 1500, ...}`: Inicializa o dicionário relacionando cada chave (nome do produto) ao seu valor (preço).

`precos_produtos["celular"]`: Busca e recupera diretamente o valor 1500 vinculado à chave "celular".

`precos_produtos["televisao"] = 3000`: Insere a chave "televisao" mapeando o valor 3000 no dicionário.

Comparativo e Boas Práticas

Estrutura	Ordenada?	Mutável?	Forma de Acesso	Quando utilizar
Lista (list)	Sim	Sim	Por índice numérico (vendas)	Coleções de dados do mesmo tipo que sofrem adições, remoções ou reordenações frequentes.
Tupla (tuple)	Sim	Não	Por índice numérico (ponto)	Coleções fixas onde a integridade dos dados deve ser preservada (ex.: retornos múltiplos).
Dicionário (dict)	Sim (Python 3.7+)	Sim	Por chave identificadora (<code>precos["celular"]</code>)	Mapeamento de rótulos/identificadores para valores, permitindo buscas diretas e rápidas.

Fontes que sustentam a resposta

`fonte_python_analise_automacao_dados.pdf`: Enumera listas, tuplas e dicionários entre os conceitos fundamentais da linguagem.

Curso Python para Iniciantes: Apresenta a criação e manipulação prática de listas, uso de dicionários para associação e aplicação de tuplas para retorno de funções e desempacotamento.

O tutorial do Python — Documentação Python 3.14.8: Seção 5 dedicada à especificação e métodos das estruturas de dados do Python.

💡 Podemos criar um pequeno exercício prático combinando listas e dicionários para simular o cadastro e a consulta de uma tabela de produtos com seus respectivos preços.